RefIMP: An IMP Language with References

FORMATIVE REPORT DRAFT | 5 NOVEMBER 2026 | NOT A FINAL SUBMISSION

Student: Rzazade Elmir **Student ID:** 3202905

1. Informal language design

RefIMP extends the imperative language IMP studied in the module. It retains integer arithmetic, mutable variables, conditionals and `while` loops, but adds *lexically scoped local variables* and *dynamically allocated references to integers*. Unlike basic IMP, which uses named storage locations directly, RefIMP distinguishes a stack of variable-binding frames from a heap of allocated integer cells. Multiple variables can hold the same reference, so mutations through one name are observable through another (aliasing).

The language is intentionally small: there are no functions, manual deallocation or garbage collection, and only integer references (not references to references). Expressions may allocate memory, so they are evaluated left-to-right. A proposed static type system distinguishes Int, Bool and Ref Int. The full assignment will accept plain-text source files; the current formative prototype executes manually constructed abstract syntax trees (ASTs).

2. Formal abstract syntax

Let $n \in \mathbb{Z}$, x range over variable identifiers, and $k, m \geq 0$. Source syntax uses braces and semicolons; the following is abstract syntax (so S is a list of statements).

T	::=	Int | Bool | Ref Int	types
e	::=	n | true | false | x | $e + e$ | $e - e$ | $e * e$	
	|	$e \leq e$ | $e == e$ | new(e) | $*e$	expressions
s	::=	let $x : T = e$ | $x := e$ | $*e := e$ | B	
	|	if e then B else B | while e do B	statements
B	::=	$\{S\}$ where $S ::= [] \mid s :: S$	block and sequences
P	::=	program($x_1, \ldots, x_m$) $\{S;$ return $e\}$	whole program

All program inputs are natural numbers interpreted as integers; the final returned expression must have type Int. A `return` is allowed only in the final position, avoiding early-return control flow. A Ref Int value is created only by new, not by arbitrary address literals.

3. Distinctive example: aliasing

```
program(m, n) {
    let p: Ref Int = new(m);
    let q: Ref Int = p;
    *q := *q + n;
    return *p;
}
```

For inputs $m = 5, n = 7$, this returns 12. Both `p` and `q` contain the same heap reference, so changing `*q` also changes the value subsequently observed through `*p`. The remaining required examples (addition, half, factorial, Fibonacci) are included in the draft repository as proposed source programs and as executable AST demos.

4. Operational semantics

Runtime values are $v ::= n \mid \mathsf{true} \mid \mathsf{false} \mid \mathsf{ref}(\ell)$. The *environment* ρ is a nonempty stack of finite variable-to-value frames; $\mathsf{lookup}(\rho, x)$ uses the nearest binding. declare inserts into the current frame, update modifies the nearest binding, and push/pop enter/leave a scope. The *heap* $H : \mathsf{Loc} \rightharpoonup \mathbb{Z}$ maps allocated locations to integers. Allocation chooses $\ell = \min(\mathbb{N} \setminus \mathrm{dom}(H))$, and there is no deallocation.

Expression evaluation is $\langle e, \rho, H\rangle \Downarrow (v, H')$; execution of a statement is $\langle s, \rho, H\rangle \Downarrow_s (\rho', H')$. The key rules below give the distinctive reference behaviour:

$$\frac{\langle e, \rho, H\rangle \Downarrow (n, H_1) \quad \ell = \min(\mathbb{N} \setminus \mathrm{dom}(H_1))}{\langle \mathsf{new}(e), \rho, H\rangle \Downarrow (\mathsf{ref}(\ell), H_1[\ell \mapsto n])} \quad \textsc{New}$$

$$\frac{\langle e, \rho, H\rangle \Downarrow (\mathsf{ref}(\ell), H_1) \quad H_1(\ell) = n}{\langle *e, \rho, H\rangle \Downarrow (n, H_1)} \quad \textsc{Read}$$

$$\frac{\langle e_1, \rho, H\rangle \Downarrow (\mathsf{ref}(\ell), H_1) \quad \langle e_2, \rho, H_1\rangle \Downarrow (n, H_2)}{\langle *e_1 := e_2, \rho, H\rangle \Downarrow_s (\rho, H_2[\ell \mapsto n])} \quad \textsc{Write}$$

The second expression in Write observes the heap produced by the first. Similarly, binary arithmetic/comparisons evaluate the left operand before the right. The usual rules for literals, variables, declarations, assignment, sequencing, blocks, branching and loops are listed in the appendix. We assume well-typed programs and valid declared-variable lookups for these rules.

5. Proposed static typing

Write $\Gamma \vdash e : T$ for "expression e has type T" in a lexical type environment Γ. Representative rules are:

$$\frac{\Gamma \vdash e : \mathsf{Int}}{\Gamma \vdash \mathsf{new}(e) : \mathsf{Ref\ Int}} \quad \text{T-New} \qquad \frac{\Gamma \vdash e : \mathsf{Ref\ Int}}{\Gamma \vdash *e : \mathsf{Int}} \quad \text{T-Read}$$

A heap write requires $\Gamma \vdash e_1 : \mathsf{Ref\ Int}$ and $\Gamma \vdash e_2 : \mathsf{Int}$; a condition in `if`/`while` must be Bool; assignments must preserve the declared variable type. Thus `*p := true` is rejected if `p : Ref Int`. This checker is *planned*, not yet implemented in the AST prototype.

6. Implementation and scope of the formative draft

The C++17 AST interpreter currently implements scopes, arithmetic/comparisons, loops, allocation, reading and heap writes. It runs five preconstructed demonstrations from the command line and has basic automated tests. The `.refimp` example files show the intended text syntax; *the prototype does not yet parse them.* Later work will implement lexing, recursive-descent parsing, the type checker, error reporting and integration tests. The interpreter mirrors the above big-step rules: expression evaluation returns a value while updating the heap, and each statement updates the runtime state.

Literature review (extended module; planned): I intend to read Jung et al., *Safe Systems Programming in Rust: The Promise and the Challenge* (CACM, 2020), suggested in the brief, and discuss the contrast between RefIMP's unrestricted aliasing and Rust's approach to memory safety. The required 2–3 paragraph review will be written after reading the paper; it is not represented as completed here.

AI use and collaboration (draft disclosure): ChatGPT assisted with brainstorming, the initial grammar and semantics, and preparation of the prototype/report draft.

Appendix A. Remaining big-step rules (working specification)

Expressions. Values of literal forms and variables do not change the heap; $\odot \in \{+, -, *, \leq, ==\}$ with $\delta_\odot$ the corresponding integer operation or comparison.

$$\frac{}{\langle n, \rho, H\rangle \Downarrow (n, H)}\ \textsc{Int} \qquad \frac{}{\langle b, \rho, H\rangle \Downarrow (b, H)}\ \textsc{Bool} \quad (b \in \{\mathsf{true}, \mathsf{false}\})$$

$$\frac{\mathsf{lookup}(\rho, x) = v}{\langle x, \rho, H\rangle \Downarrow (v, H)}\ \textsc{Var}$$

$$\frac{\langle e_1, \rho, H\rangle \Downarrow (n_1, H_1) \quad \langle e_2, \rho, H_1\rangle \Downarrow (n_2, H_2)}{\langle e_1 \odot e_2, \rho, H\rangle \Downarrow (\delta_\odot(n_1, n_2), H_2)}\ \textsc{Bin}$$

Statements. The rules assume the declared variable exists for assignment and does not already exist in the current frame for declaration; the runtime value matches the declared type.

$$\frac{\langle e, \rho, H\rangle \Downarrow (v, H_1)}{\langle \mathsf{let}\ x : T = e, \rho, H\rangle \Downarrow_s (\mathsf{declare}(\rho, x, v), H_1)}\ \textsc{Let}$$

$$\frac{\langle e, \rho, H\rangle \Downarrow (v, H_1)}{\langle x := e, \rho, H\rangle \Downarrow_s (\mathsf{update}(\rho, x, v), H_1)}\ \textsc{Assign}$$

$$\frac{}{\langle [], \rho, H\rangle \Downarrow_* (\rho, H)}\ \textsc{Empty}$$

$$\frac{\langle s, \rho, H\rangle \Downarrow_s (\rho_1, H_1) \quad \langle S, \rho_1, H_1\rangle \Downarrow_* (\rho_2, H_2)}{\langle s :: S, \rho, H\rangle \Downarrow_* (\rho_2, H_2)}\ \textsc{Seq}$$

$$\frac{\langle S, \mathsf{push}(\rho), H\rangle \Downarrow_* (\rho_1, H_1)}{\langle \{S\}, \rho, H\rangle \Downarrow_s (\mathsf{pop}(\rho_1), H_1)}\ \textsc{Block}$$

Conditionals and loops. Note that the guard can change the heap when evaluating subexpressions containing new.

$$\frac{\langle e, \rho, H\rangle \Downarrow (\mathsf{true}, H_1) \quad \langle B_1, \rho, H_1\rangle \Downarrow_s (\rho_1, H_2)}{\langle \mathsf{if}\ e\ \mathsf{then}\ B_1\ \mathsf{else}\ B_2, \rho, H\rangle \Downarrow_s (\rho_1, H_2)}\ \textsc{If-T}$$

$$\frac{\langle e, \rho, H\rangle \Downarrow (\mathsf{false}, H_1) \quad \langle B_2, \rho, H_1\rangle \Downarrow_s (\rho_1, H_2)}{\langle \mathsf{if}\ e\ \mathsf{then}\ B_1\ \mathsf{else}\ B_2, \rho, H\rangle \Downarrow_s (\rho_1, H_2)}\ \textsc{If-F}$$

$$\frac{\langle e, \rho, H\rangle \Downarrow (\mathsf{false}, H_1)}{\langle \mathsf{while}\ e\ \mathsf{do}\ B, \rho, H\rangle \Downarrow_s (\rho, H_1)}\ \textsc{While-F}$$

$$\frac{\langle e, \rho, H\rangle \Downarrow (\mathsf{true}, H_1) \quad \langle B, \rho, H_1\rangle \Downarrow_s (\rho_1, H_2) \quad \langle \mathsf{while}\ e\ \mathsf{do}\ B, \rho_1, H_2\rangle \Downarrow_s (\rho_2, H_3)}{\langle \mathsf{while}\ e\ \mathsf{do}\ B, \rho, H\rangle \Downarrow_s (\rho_2, H_3)}\ \textsc{While-T}$$

Whole-program evaluation. Let ρ_0 be the initial frame containing integer inputs $x_i \mapsto n_i$, and let $H_0 = \varnothing$. For $P = \mathsf{program}(x_1, \ldots, x_m)\{S; \mathsf{return}\ e\}$:

$$\frac{\langle S, \rho_0, H_0\rangle \Downarrow_* (\rho_1, H_1) \quad \langle e, \rho_1, H_1\rangle \Downarrow (n, H_2)}{P(n_1, \ldots, n_m) \Downarrow n}\ \textsc{Program}.$$